

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis

Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

- The likely design error is **poor input data partitioning**. If input files or blocks are highly uneven, some Mapper tasks receive much more data than others. As a result, some tasks take longer while other tasks finish early. This can also cause Reducers to remain idle.
- To correct this issue, the input data should be divided into more balanced splits. The **HDFS block size and input split configuration** should be reviewed. Small files should be combined where possible, and large files should be partitioned appropriately. The number of Mapper and Reducer tasks should also be adjusted according to the data size. Using a **Combiner** can reduce intermediate data and improve performance.
- Therefore, balanced input splits and proper task configuration can prevent task imbalance and improve Hadoop job execution.

2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

- Frequent data unavailability after a single DataNode failure usually indicates an incorrect **HDFS replication configuration** or a problem with **NameNode availability**.
- If the replication factor is set to 1, a block stored only on the failed DataNode becomes unavailable. The replication factor should normally be increased to **2 or 3**, depending on the reliability requirements. HDFS should distribute replicas across different DataNodes and racks.
- If the NameNode itself is a single point of failure, **NameNode High Availability (HA)** should be configured with Active and Standby NameNodes. Proper monitoring should also be enabled so that failed nodes and under-replicated blocks are detected quickly.
- Thus, sufficient replication and NameNode HA are essential for maintaining data availability during node failures.

3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

- Duplicate records can occur when the ingestion pipeline processes the same source data multiple times. Common causes include **repeated file processing, incorrect NiFi flow design, retry operations, and lack of unique record identification**.
- To solve this problem, NiFi should track processed files and use suitable processors and flow control mechanisms. A unique ID or business key should be assigned to each record. The analytics database should also use **primary keys, unique constraints, or deduplication logic**.
- For database sources, incremental loading or **Change Data Capture (CDC)** can be used instead of repeatedly loading the complete dataset. Proper error handling and retry configurations should also prevent successful records from being inserted again.
- Therefore, source tracking, unique identifiers, incremental ingestion, and destination-side deduplication can effectively reduce duplicate records.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

- Resource starvation occurs when the YARN scheduler allows one or a few applications to consume most of the available CPU and memory. This indicates an unsuitable **scheduling or queue configuration**.
- A better approach is to use **Capacity Scheduler or Fair Scheduler** to divide resources among different users or application queues. Resource limits can be configured for each queue so that one user cannot consume the entire cluster.
- Priority can be given to important jobs while still reserving resources for other users. User and application limits should also be configured, and YARN monitoring should be used to identify resource-intensive jobs.
- Therefore, proper queue allocation, resource limits, scheduling policies, and monitoring can prevent starvation and provide fair resource sharing.

5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

- Restoring outdated data usually indicates that the backup system is not maintaining **recent backup versions or synchronization** correctly. The backup may have been performed too infrequently, or the system may be restoring an old snapshot instead of the latest valid backup.
- To correct this problem, regular **incremental and full backups** should be scheduled. Backup timestamps and version information should be maintained so that the most recent valid copy can be identified. Replicas should also be synchronized properly, and backup integrity should be verified periodically.
- A suitable retention policy should maintain multiple recovery points instead of depending on a single backup. Recovery testing should also be performed regularly.
- Thus, frequent backups, version control, synchronization, and regular recovery testing can ensure that the latest available data is restored after a failure.